

Tech Challenge

Fase 3

Oficina FIAP — arquitetura em nuvem, autenticação serverless, CI/CD e observabilidade

GRUPO	TURMA	DATA
267	SOAT15	14 de setembro de 2026

Documento único de entrega

Este relatório reúne os links e as evidências arquiteturais solicitadas no enunciado oficial. Os detalhes de execução e implantação permanecem nos READMEs de cada repositório.

Identificação e links de entrega

Acesso rápido aos artefatos obrigatórios

Equipe

Nome completo	RM	Usuário do Discord
José Pedro Pisano Filho	RM371010	pisano7196
Gabriel Freitas Souza	RM371657	fritas / Gabriel Souza
Gabriel Ramos de Sousa	RM373673	squarecondition
Guilherme Farias Menoci	RM372504	menoci

Repositórios

ENTREGÁVEL	REPOSITÓRIO	RESPONSABILIDADE
Aplicação principal	oficina_fiap	Spring Boot, domínio, Docker, manifests, deploy e observabilidade
Infraestrutura Kubernetes	oficina-fiap-infra-k8s	Terraform para EC2, rede, Security Group e K3s
Infraestrutura de banco	oficina-fiap-infra-database	Terraform para Amazon RDS PostgreSQL
Autenticação serverless	oficina-fiap-lambda-auth	API Gateway, Lambda CPF, Authorizer e Terraform

Vídeo de demonstração

VÍDEO DISPONÍVEL

[Abrir vídeo de demonstração no Google Drive](#)

Google Drive

Links complementares

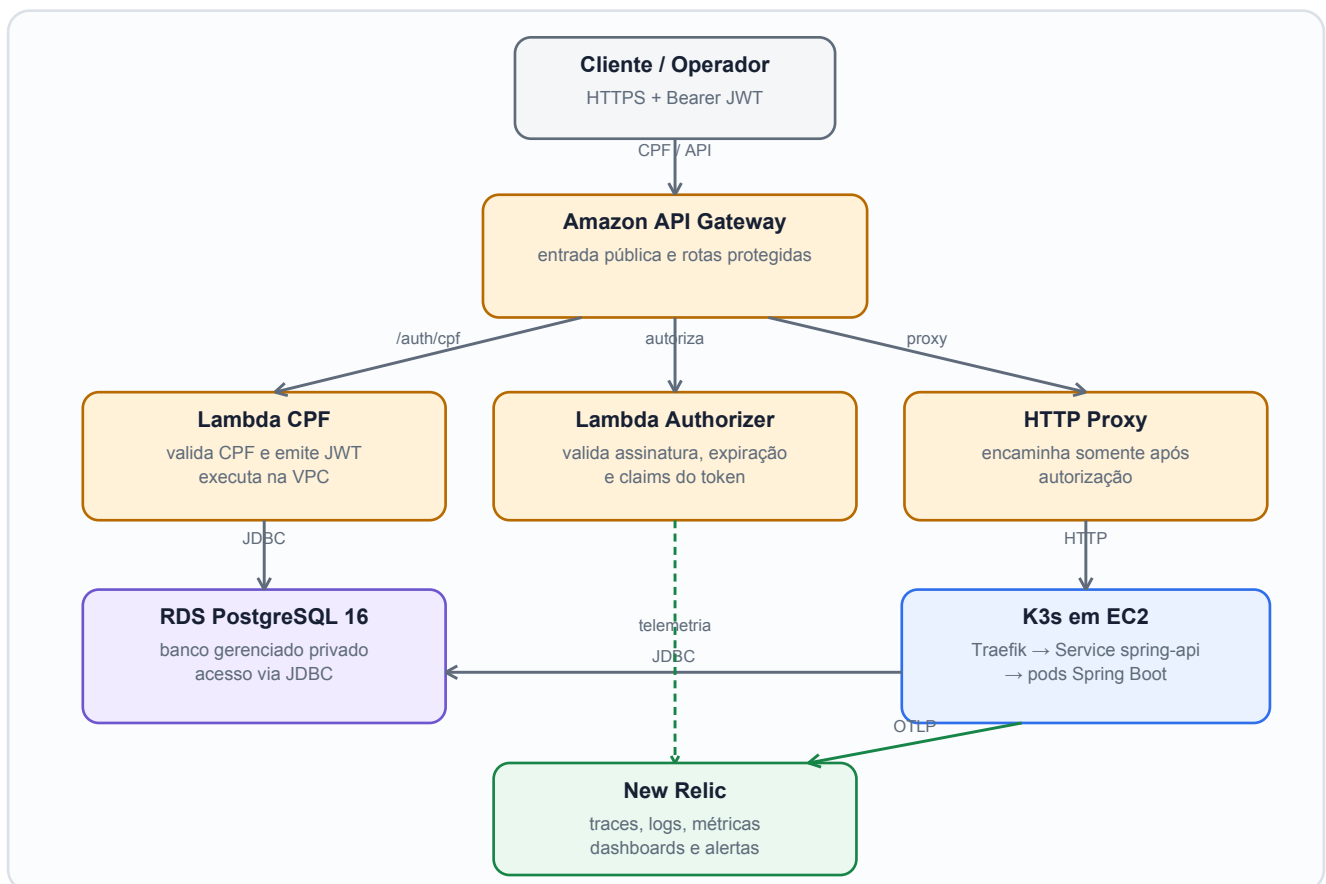
Documento	Acesso
Arquitetura e implantação	Abrir documentação consolidada
RFCs e ADRs	Abrir índice de decisões
Coleção Postman	Abrir coleção da API
Dashboard New Relic	Abrir dashboard (requer acesso à conta)

Resumo da solução

Como os componentes atendem ao desafio

A Oficina FIAP foi evoluída para uma arquitetura em nuvem com quatro ciclos de entrega independentes. O acesso de clientes começa no API Gateway, a autenticação por CPF acontece em uma Lambda conectada ao banco privado e as rotas sensíveis são protegidas por um Lambda Authorizer. A aplicação Spring Boot é executada em Kubernetes K3s e persiste os dados em PostgreSQL gerenciado no Amazon RDS.

A plataforma New Relic centraliza traces, métricas, logs estruturados, dashboards e alertas. GitHub Actions automatiza testes, empacotamento e deploy; Terraform provisiona cluster, banco e componentes serverless, enquanto a aplicação é atualizada no K3s por kubectl e a integração de observabilidade é instalada por Helm.



Autenticação e proteção das APIs

Fluxo validado de ponta a ponta na AWS

Fluxo de emissão do token

- O cliente envia seu CPF para POST /auth/cpf no Amazon API Gateway.
- A Lambda de autenticação valida o formato do CPF e consulta o cliente no RDS privado.
- Para um cliente existente e ativo, a função emite um JWT com validade de 3.600 segundos.

Fluxo de uma rota protegida

#	Componente	Ação
1	Cliente	Envia Authorization: Bearer <JWT> ao API Gateway.
2	Lambda Authorizer	Valida assinatura, expiração e claims; retorna Allow ou Deny.
3	API Gateway	Com Allow, encaminha /api/{proxy+} para APP_BASE_URL.
4	Traefik / K3s	Ingress recebe HTTP na porta 80 e direciona ao Service spring-api:8080.
5	Spring API	Valida novamente o JWT e aplica a autorização do cliente.

Controles adotados

Controle	Aplicação
Segredo compartilhado	O mesmo JWT_SECRET é disponibilizado às Lambdas e à Spring API por secrets dos repositórios.
Defesa em profundidade	O Authorizer bloqueia no Gateway e a aplicação também valida o token enquanto o backend puder ser acessado diretamente.
Banco privado	A Lambda executa na VPC e acessa o RDS por JDBC sem torná-lo público.
Expiração	JWT de curta duração reduz o período de exposição de um token comprometido.

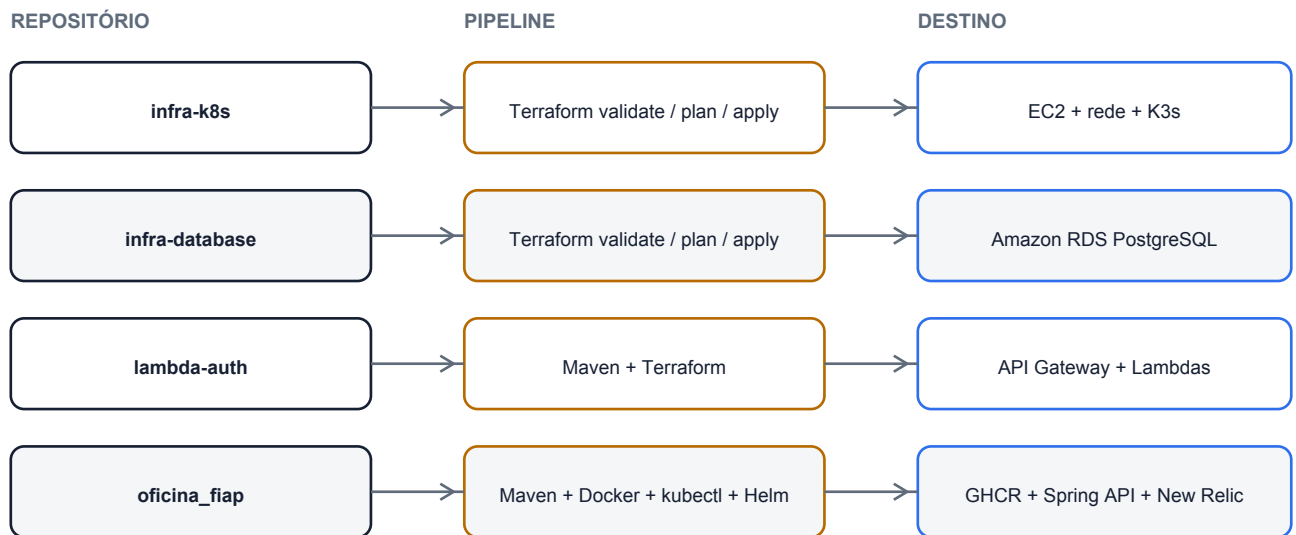
Evidência funcional

O fluxo CPF → JWT → Authorizer → API Gateway → Spring API foi executado com cliente ativo e resposta real da rota protegida.

Infraestrutura e CI/CD

Quatro repositórios, quatro responsabilidades

O desenho separa aplicação, cluster, banco e autenticação para que cada parte tenha estado, credenciais e frequência de mudança próprios. A integração ocorre por contratos explícitos: endpoint do RDS, dados de rede, APP_BASE_URL e JWT_SECRET.



Branches protegidas e alterações integradas por Pull Request; credenciais e chaves ficam em GitHub Secrets/Variables.

Estratégias de deploy

Repositório	Validações e entrega	Execução
infra-k8s	terraform fmt, validate, plan e apply	Runner Ubuntu / AWS
infra-database	terraform fmt, validate, plan e apply	Runner Ubuntu / AWS
lambda-auth	build/test Java e Terraform para API Gateway/Lambdas	Runner Ubuntu / AWS
oficina_fiap	Maven, testes, imagem GHCR, kubectl e Helm	Runner Windows self-hosted

Controles de entrega

- Alterações integradas na branch principal por Pull Request.
- Branches principais protegidas e pipelines associados a cada repositório.
- Credenciais AWS, banco, JWT e New Relic mantidas em GitHub Secrets/Variables.
- Estados Terraform separados em backend S3, reduzindo o raio de impacto.

Atenção operacional

O IP público da EC2 pode mudar quando o laboratório é recriado. Nesse caso, o APP_BASE_URL do deploy serverless precisa ser atualizado antes do teste ponta a ponta.

Banco de dados gerenciado

PostgreSQL 16 no Amazon RDS

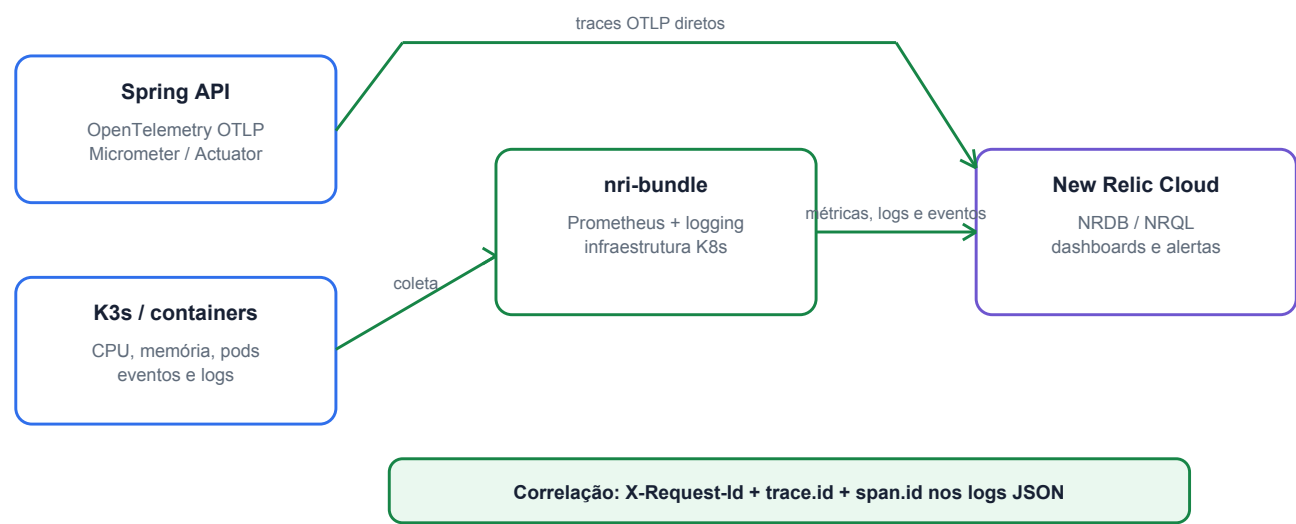
A aplicação já possuía um modelo relacional e operações transacionais. O PostgreSQL no Amazon RDS preserva essa compatibilidade e separa o ciclo de vida dos dados do cluster K3s. O endpoint não é público e a porta 5432 é permitida somente dentro da rede definida para o ambiente.

Justificativas e configuração

Aspecto	Decisão	Motivo
Tecnologia	PostgreSQL 16	Compatível com JDBC/JPA, modelo relacional e transações do domínio.
Gerenciamento	Amazon RDS	Dados independentes dos pods e da recriação da instância EC2.
Rede	Endpoint privado	Reduz a exposição; aplicação e Lambda acessam o banco pela VPC.

Observabilidade

Métricas, logs, traces, healthchecks e alertas



Requisito	Implementação	Evidência esperada
Latência das APIs	Spans OpenTelemetry exportados por OTLP	Widget de latência p95 e traces do serviço tc-oficina
CPU e memória K8s	newrelic-infrastructure / nri-bundle	Uso, requests e limits por pod
Health e uptime	Actuator + probes liveness/readiness + disponibilidade de pods	Pods desejados/disponíveis e endpoints de health; Synthetic externo é evolução recomendada
Falhas de ordens	Micrometer e condição NRQL	Alerta disparado por erro HTTP controlado
Logs correlacionados	JSON + RequestCorrelationFilter	requestId, trace.id e span.id

Dashboard Oficina FIAP — Observabilidade

- ☐ Volume diário de ordens de serviço
- ☐ Erros HTTP e falhas de processamento
- ☐ Memória usada, reservada e limite
- ☐ Pods desejados, disponíveis e indisponíveis
- ☐ Tempo médio por status
- ☐ Latência p95
- ☐ CPU/memória em relação aos limites

Decisões arquiteturais

RFCs e ADRs consolidados

As RFCs registram as propostas discutidas e os ADRs preservam decisões aceitas, alternativas e consequências. O índice completo está no diretório de decisões do repositório principal.

REGISTRO	DECISÃO	ESTADO
RFC-001	Autenticação por CPF, JWT e API Gateway	Aprovada, implementada e validada
RFC-002	Exposição da Spring API para o API Gateway	Aprovada, implementada e validada
ADR-001	Kubernetes K3s em instância EC2	Aceita e implementada
ADR-002	PostgreSQL 16 no Amazon RDS	Aceita e implementada
ADR-003	Escalabilidade horizontal com HPA	Aceita e implementada
ADR-004	New Relic, OpenTelemetry e nri-bundle	Aceita, implementada e validada
ADR-005	Quatro repositórios e pipelines independentes	Aceita e implementada

Principais trade-offs documentados

K3s em nó único	Menor custo e operação simples; sem alta disponibilidade do control plane.
RDS privado Single-AZ	Banco gerenciado e isolado; disponibilidade limitada pelo perfil acadêmico.
JWT validado duas vezes	Defesa em profundidade enquanto o backend permanece publicamente acessível.
Quatro repositórios	Responsabilidades e estados isolados; exige coordenação de secrets e endpoints.
New Relic SaaS	Telemetria centralizada; depende de conectividade e limites da conta.

Rastreabilidade

[Consultar o índice completo de RFCs e ADRs](#) para contexto, alternativas, consequências e evidências de cada decisão.